

LFX Mentorship 2026 Term 2 — urunc Monitor Lifecycle

Raman Kapoor

11 May 2026

LFX Mentorship 2026 Term 2 — Proposal

Project: CNCF / urunc — Improve lifecycle management of sandbox monitors **Upstream issue:** [urunc-dev/urunc#112](https://github.com/cncf/urunc-dev/issues/112) **LFX project:** <https://mentorship.lfx.linuxfoundation.org/project/02b6f81b-a96d-4fc5-894d-3da12a3564bf> **Mentors:** Charalampos Mainas (@cmainas), Anastassios Nanos (@ananos) **Applicant:** Raman Kapoor — GitHub [@unchangedraman](https://github.com/unchangedraman), unchangedraman@gmail.com **B.Tech IT, ABV-IIITM Gwalior (graduating May 2026)**

1. About me

I am a final-year B.Tech (Information Technology) student at IIITM Gwalior, currently a Software Developer intern at Juspay (India's largest payments orchestration platform). My day-to-day work is in low-level infrastructure: I led the replacement of our CEPH block-storage tier with Lightbits NVMe/TCP — same workload at 48K TPS vs 37K TPS on roughly a tenth of the hardware (4 nodes vs 40), and active-active Barbican HA on a shared backend. I also shipped a thread-local + server-local cache layer in Rust on hot APIs that cut cross-node calls by ~4% at peak, and a Shamir-based secrets distribution layer with memory-only keys and Raft replication that removed our reliance on AWS KMS.

Before Juspay I built a small 32-bit i386 kernel from scratch in C++ — bootstrapping, PIC + ISR/IRQ wiring, GDT, the usual — because I wanted to understand what a “process” really is on hardware. That project is where my interest in monitors and hypervisors started.

Between Nov 2025 and Jan 2026 I worked as an open-source backend engineer on [Namma Yatri](#), the open-source ride-hailing platform that runs in production across India. My main piece there was [nammayatri/nammayatri#13442](#) — a distributed, event-driven payout scheduling system for special-zone rides, serving roughly 10,000 rides per day, which cut driver payout latency from weekly cycles down to T+2 hours. The interesting part was the Redis-backed delayed-job scheduler with per-ride locks and idempotent state transitions; that piece had to be exactly-once under production traffic without adding latency to the ride-booking hot path. The follow-up PR [nammayatri/nammayatri#13452](#) surfaced the resulting payout state to the driver-app frontend. I also shipped a separate [PaytmEDC payment integration in their shared kernel](#). All three are merged. I mention this because it is the closest thing I have to “shipped upstream code in a large unfamiliar Haskell codebase with maintainer review” — which is the muscle this LFX project will need.

I have been working with urunc in my home lab for the past few weeks. Issue [urunc-dev/urunc#389](#) (high host CPU when consoles are active) is mine, and is where I started talking to Charalampos and Anastassios. I wrote a small KVM-exit + throughput sampler ([bench-console.sh](#)) and traced the regression to per-byte PIO traps on the 16550 UART; the baseline serial stdio configuration was costing roughly 1.16 M PIO exits per 5 s window while a simple virtio-console swap drops that to ~0 and lifts throughput from 2,016 to 73,378 lines/sec (about 36x). That whole thread shaped how I think about urunc’s monitor surface and is part of why this project is the one I want to work on.

2. Why this project

urunc today shells out to QEMU or Firecracker with a long `-x ... -y ...` command string. That is enough to start a sandbox, but everything after that — checking what the guest is doing, hotplugging a device, asking the monitor to gracefully shut down, attaching a console after the fact — is either impossible or has to be approximated by signalling the host process and hoping for the best.

Both QEMU and Firecracker already expose a much better interface for this: QEMU has the QEMU Machine Protocol (QMP, line-delimited JSON over a Unix socket) and the older Human Monitor Protocol, and Firecracker exposes a REST-style API socket. Issue [#112](#) is exactly this gap — urunc has CLI for spawn but no socket-based control plane for the lifecycle that follows.

The cost of not having this surfaces in real places: pause/resume isn’t really possible, hotplug isn’t really possible, and the urunc-side state machine has to infer monitor state from process exit

codes rather than asking the monitor directly. It also means each new monitor we add (cloud-hypervisor, Dragonball, etc.) is a fresh integration with no shared abstraction.

I see this project as two pieces that have to land together:

1. **A monitor-agnostic control interface inside urunc** — an internal Go interface that captures the operations urunc actually needs (probe, query state, send shutdown, hotplug net/block, attach console). The existing VMM interface in `pkg/unikontainers/types/types.go` is a sensible home for this; right now it carries `BuildExecCmd`, `Stop`, `Path`, `UsesKVM`, `SupportsSharedfs`, `Ok`, and a `PreExec` for HVT seccomp. The new surface would extend it without breaking those.
2. **Per-monitor implementations** that translate the interface into QMP for QEMU and into the Firecracker API for Firecracker, plus a small fallback path for monitors that don't expose a socket (so the existing CLI-only flow keeps working).

This matches the two expected outcomes in the project description: a design document and the implementation.

3. Approach

I want to be careful here, because urunc's current design intentionally keeps the monitor-process model simple — `BuildExecCmd` returns an `argv` that gets `syscall.Exec'd`, and `Stop` just kills the resulting PID. A socket-based control plane changes that shape. So my plan is to not delete the CLI path; instead, to add a control channel alongside it that becomes the default when the monitor supports it.

3.1 Sketch of the design

- Add a `MonitorControl` interface (working name) under `pkg/unikontainers/hypervisors/` with methods for the operations urunc actually needs today, plus the ones #112 explicitly mentions (state query, hotplug, guest interaction).
- For QEMU, generate `-qmp unix:<sock>,server,nowait` as part of the existing `BuildExecCmd` flow. After spawn, the sequence is: read the greeting, send `qmp_capabilities`, then any of the operational commands the interface exposes. Concretely, I would implement these QMP commands in order of project priority — `query-status` (probe + state machine), `quit` (graceful shutdown to replace `killProcess`), `stop/cont` (pause/resume), `device_add/device_del` (hotplug for `virtio-net` and `virtio-blk` first), `chardev-add/chardev-remove` (console attach detach), and `query-vcpus/query-block` for introspection. JSON over the socket is small enough that I'd rather hand-roll the wire types than pull in a heavy QMP library; I'll evaluate github.com/digitalocean/go-qmp.

qemu and a couple of alternatives in the design doc and recommend one with reasons.

- For Firecracker, the existing `firecracker-go-sdk` already wraps the API socket; the work is mostly wiring `urunc`'s interface methods to the SDK's `PauseVM` / `ResumeVM` / `PatchDrive` / `PatchMMDS` / `GetMachineConfig` calls, and deciding which subset we actually expose at the `urunc` layer. Firecracker's API surface is smaller than QEMU's, so the `MonitorControl` interface needs to be expressive enough for QMP without forcing Firecracker to fake operations it doesn't support — capability flags on the interface, the same shape `urunc` already uses for `SupportsSharedfs`.
- Socket lifecycle is the tricky part: where the socket lives (per-container subdir under `urunc`'s state dir is the natural choice), who cleans it up, and what happens when the monitor dies between us opening the socket and us using it. I'll work the failure modes out in the design doc before writing code.
- Keep the CLI-only fallback for monitors without a control socket (HVT and similar) so this is purely additive.

3.2 What I would explicitly not do

- I won't rip out the existing `BuildExecCmd` / `syscall.Exec` shape. The new socket-based control is layered on top, not a replacement.
- I won't try to also rewrite the `seccomp` handling (`PreExec` for HVT) or the monitor-rootfs work in the same project. Those are adjacent but big enough to be their own conversations.
- I won't add a new monitor (cloud-hypervisor, Dragonball, etc.) inside the LFX scope. The interface should make it easy to add one later — that's a different person's first PR.
- I won't introduce a new dependency for QMP unless the design doc justifies it.

3.3 What I would prove out before merging

- The QMP-controlled QEMU path can spawn, query state, gracefully shut down, and hotplug a `virtio-net` device end-to-end, with the existing `nginx-on-unikraft` integration tests still green.
- The Firecracker path can do the same subset.
- A simple benchmark — same workload as my #389 work, but measuring monitor-control overhead (number of round-trips, socket I/O cost) — so we have a number for the design doc and not just "it works."
- The fallback path (CLI-only, no socket) still works for any current monitor.

4. Weekly plan (12 weeks, 8 Jun - 31 Aug 2026)

The LFX Term 2 program runs 8 Jun - 31 Aug 2026. I treat the first two weeks as design-only and the last two as buffer + write-up, which leaves eight weeks for code.

Week	Dates	Focus	Concrete deliverable
1	Jun 8 - Jun 14	Onboarding + read	Re-read all of pkg/unikontainers/hypervisors/, write up a short “current state of monitor lifecycle in urunc” gist for the mentors to correct.
2	Jun 15 - Jun 21	Design doc, draft 1	Markdown design doc in docs/ proposing the MonitorControl interface, QMP + Firecracker mapping, socket lifecycle, fallback behaviour. Open as a draft PR for review.
3	Jun 22 - Jun 28	Design doc, revisions	Address review comments; settle the interface signature and dependency decision. Tag the doc as “ready” once mentors sign off.
4	Jun 29 - Jul 5	QEMU/QMP, minimum viable	Wire -qmp into the existing QEMU BuildExecCmd, implement capability handshake + query-status + quit. End of week: a urunc-launched QEMU sandbox can be queried and gracefully stopped via QMP.
5	Jul 6 - Jul 12	QEMU/QMP, full lifecycle	cont/stop (pause/resume), device_add/device_del for virtio-net and virtio-blk, error handling for unsupported commands. Unit tests for the QMP wire codec.
6	Jul 13 - Jul 19	Firecracker API	Implement the same MonitorControl surface against firecracker-go-sdk. Pause/resume, drive patch, network patch where applicable.
7	Jul 20 - Jul 26	Integration	Plumb the new control path through unikontainers.go so urunc’s own state machine

Week	Dates	Focus	Concrete deliverable
8	Jul 27 - Aug 2	Tests + CI	uses it. Make sure the CLI-only fallback still works on a monitor that doesn't expose a socket. Add integration tests under the existing test layout. Add a small bench script (extension of bench-console.sh) that times the QMP round-trip cost. CI workflow update.
9	Aug 3 - Aug 9	Stretch / hotplug demo	End-to-end hotplug demo: start a unikraft nginx sandbox, hotplug a second virtio-net device, verify connectivity. Document the recipe.
10	Aug 10 - Aug 16	Bug-fix + docs	Address review comments on the implementation PRs. Write user-facing docs (how to talk to a urunc sandbox's QMP socket, how to opt into the new control path).
11	Aug 17 - Aug 23	Buffer	Reserved for slippage. If none, start on a follow-up issue from the mentors' backlog.
12	Aug 24 - Aug 31	Wrap-up + blog	Final PR clean-up, mentee blog post for the CNCF / Nubificus blog covering the architecture and the numbers.

I expect to break Weeks 4-7 into multiple small PRs rather than one large one, so review can happen in parallel with development.

5. Why I think I am a fit

- **I have already contributed to urunc.** Issue #389 has been ongoing for a few weeks with cmainas and ananos; my last comments included a bench harness and an empirical PIO-exit measurement. The maintainers know my work.
- **I have shipped upstream before.** Three merged PRs in the [Namma Yatri](#) open-source ride-hailing org, including a distributed event-driven payout scheduler ([#13442](#)) running in production. I can navigate a large unfamiliar codebase, take review comments, and get code merged.

- **I work in Go and Rust on Linux infrastructure full-time.** Juspay's storage migration that I led (CEPH → Lightbits NVMe/TCP, 40 nodes → 4) and the Rust caching + Go secrets-broker work I shipped there are exactly the kind of `syscall.Exec` / process-management / Unix-socket code this project needs.
- **Low-level systems is where I want to be.** The BREAK OS kernel project (32-bit i386, GDT, PIC, ISRs, all from scratch in C++) is what got me into this; this LFX project is essentially the userspace side of the same world.
- **I can debug empirically.** The #389 work involved tracing per-byte PIO exits via `/sys/kernel/debug/kvm/io_exits` rather than guessing — I expect the QMP work to need the same posture, especially around socket lifecycle edge cases.
- **CNPG + Cluster API project.** My most recent independent project provisions HA Postgres on Kubernetes-on-OpenStack with CAPO + CloudNativePG, RPO=0 and ~7 s failover. It is not directly urunc, but it is the same shape of work: API-driven lifecycle of stateful infrastructure on top of VMM-based isolation.

5.1 Work I am doing before applications close

The application window closes May 19. Between now and then, my plan is to land the following so the mentors can see the work, not just read the plan:

- A small standalone Go program in this repo that opens a Unix socket to a `qemu-system-x86_64` instance, sends `qmp_capabilities`, then `query-status` and `quit`. Roughly 100 lines, intentionally throwaway — just to prove the wire shape and identify the rough edges.
- A second, equally small Go program against the Firecracker API socket using `firecracker-go-sdk`, doing `GetMachineConfig` and `PauseVM/ResumeVM` on a stopped microVM.
- A 1-page sketch in this repo (`docs/design-sketch.md`) of what the `MonitorControl` interface signature would look like and where it would slot into `pkg/unikontainers/hypervisors/types.go`. Not a finished design — a starting point for the actual design doc in week 2.

This is not part of the LFX commitment; it is what I am doing this week regardless, because the only way to know whether the design is right is to write a little of it.

6. Standard LFX cover-letter questions

How did you find out about the mentorship program? I follow CNCF projects on GitHub and was reading the 2026 Term 2 project ideas list when I saw the urunc entries. urunc itself I had already been using locally because the unikernel-as-container model is interesting to me, and a few weeks ago I filed urunc-dev/urunc#389 against it.

Why are you interested in this program? I want to spend three months working on a single piece of cloud-native infrastructure at depth, with maintainers reviewing my code, instead of the bouncing-around pattern that drive-by open-source contributions tend to have. urunc specifically is the right size of project for that: small enough that I can hold the whole codebase in my head, and load-bearing enough that the work matters.

What experience and knowledge / skills do you have that are applicable? Go (Juspay infrastructure, side projects, urunc work to date), Linux systems programming (the kernel project, my Juspay caching work, urunc debugging on KVM), familiarity with VMMs (QEMU on KVM is what my whole urunc lab runs on, Firecracker I have used through the firecracker-go-sdk for a smaller project), and container/OCI internals (some runc reading + the urunc debugging gave me the basics). On the open-source-collaboration side, my three merged Namma Yatri PRs (Haskell, ~10K rides/day in production) cover the part of this that is not about languages — reading an unfamiliar codebase, taking review, iterating to merge. Resume attached separately.

What do you hope to get out of this mentorship? - A real piece of upstream urunc that I designed and shipped end-to-end. - Code-review feedback from cmainas and ananos at the volume you only get when you are actively working on a maintainer's project. - Enough fluency in the urunc internals to keep contributing after the program ends.

7. Availability and commitment

I graduate from IIITM Gwalior in May 2026, so the LFX program (Jun 8 – Aug 31) sits cleanly between graduation and any later commitments. My current Juspay internship ends in May. I am committing the program as a full-time effort (40 hours/week) — I will not be juggling other engagements during it. Time zone is IST (UTC+5:30); I will overlap with Athens (UTC+2/+3) for roughly 5 hours every working day, which I have already been doing for the #389 thread.

8. Links

- Prior urunc contribution: [urunc-dev/urunc#389](https://github.com/urunc-dev/urunc/pull/389)
 - This proposal repo: <https://github.com/unchangedraman/proposal>
 - Merged open-source contributions (Namma Yatri):
 - [nammayatri/nammayatri#13442](https://github.com/nammayatri/nammayatri/pull/13442) — Special-zone payout system with history (Haskell, Redis, Postgres; ~10K rides/day)
 - [nammayatri/nammayatri#13452](https://github.com/nammayatri/nammayatri/pull/13452) — Driver payout status exposed to frontend
 - [nammayatri/shared-kernel#1108](https://github.com/nammayatri/shared-kernel/pull/1108) — PaytmEDC payment flow
 - GitHub: <https://github.com/unchangedraman>
 - LinkedIn / résumé: [to be added on the LFX form]
-

Last updated: 11 May 2026.